

Introduction to the ARM Instruction SetIntroduction:

The ARM7 architecture, programmers model and operating modes we need to take a look at its instruction set or rather sets. Since all our programming examples are written in C there is no need to be an expert ARM7 assembly programmer. However an understanding of the underlying machine code is very important in developing efficient programs. Before we start our overview of the ARM7 instructions it is very important to set out a few technologies & technicalities. The ARM7 CPU has two instruction sets: the ARM instruction set which has 32-bit wide instructions and the THUMB instruction set which has 16-bit wide instructions. In the following section the use of word ARM means the 32-bit instruction set.

One of the most interesting features of the ARM instruction set is that every instruction may be conditionally executed. In a more traditional microcontroller the only conditional instructions are conditional branches and maybe a few others like bit test and set. However in the ARM instruction set the top four bits of the operand are compared to the condition codes in the CPSR. If they do not match then the instruction is not executed and passes through the pipeline as a NOP (no operation).

Conditional Execution:

Every ARM (32 bit) instruction is conditionally executed. The top four bits are ANDed with the CPSR condition codes. If they do not match the instruction is executed as a NOP so it is possible to perform a data

processing instruction, which affects the condition codes in the CPSR. Then depending on this result, the following instructions may or may not be carried out. The basic assembler instructions such as MOV or ADD can be prefixed with sixteen conditional mnemonics, which define the condition code states to be tested for.

Each ARM (32-bit) instruction can be prefixed by one of 16 condition codes. Hence each instruction has 16 different variants.

Conditional Codes:

The condition codes and use are shown in table. If the condition is omitted in instructions, the AL(always) condition is used to specify that the instruction should always execute.

Opcode [31:28]	Mnemonic Extension	Meaning	Conditional flag state
0000	EQ	Equal	$Z == 1$
0001	NE	Not Equal	$Z == 0$
0010	CS/HS	Carry set / Unsigned Higher or same	$C == 1$
0011	CC/LO	Carry clear / Unsigned lower	$C == 0$
0100	MI	Minus / Negative	$N == 1$
0101	PL	Plus / Positive or zero	$N == 0$
0110	VS	Overflow	$V == 1$
0111	VC	No overflow	$V == 0$

1000	HI	Unsigned Higher	$(C==1) \text{ AND } (Z==0)$
1001	LS	Unsigned lower or same	$(C==1) \text{ OR } (Z==0)$
1010	GE	signed Greater than or Equal	$N==V$
1011	LT	signed Less than	$N \neq V$
1100	GT	signed Greater than	$(Z==0) \text{ AND } (N==V)$
1101	LE	signed Less than or Equal	$(Z==1) \text{ OR } (N \neq V)$
1110	AL	Always (unconditional)	Not applicable
1111	NV	Never	Absolute

Table: Condition codes and their meanings.

Instruction Set :

The main instruction groups of the ARM instruction set fall into six different categories. Branching, Data processing, Data Transfer, Block Transfer, Multiply and Software Interrupt.

Data Processing Instructions :

The data processing instructions manipulate data within registers. They are move instructions, arithmetic instructions, logical instructions, comparison instructions, and multiply instructions. Most data processing instructions can process one of their operands using the barrel shifter.

If you use the S suffix on a data processing instructions, then it updates the flags in the cpsr. Move and logical operations update the carry flag C, negative flag N, and zero flag Z. The carry flag is set from the of the barrel shift as the last bit shifted out. The N flag is set to bit 31 of the result. The Z flag is set if the result is zero.

The general form for all data processing instructions is shown fig below. Each instruction has a result register and two operands. The first operand must be a register, but the second can be a register or an immediate value.

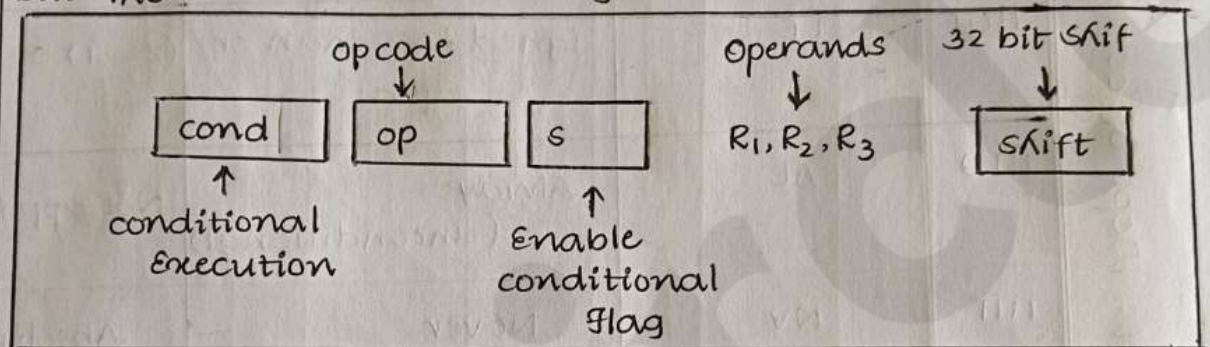


Fig: General structure of data processing instructions.

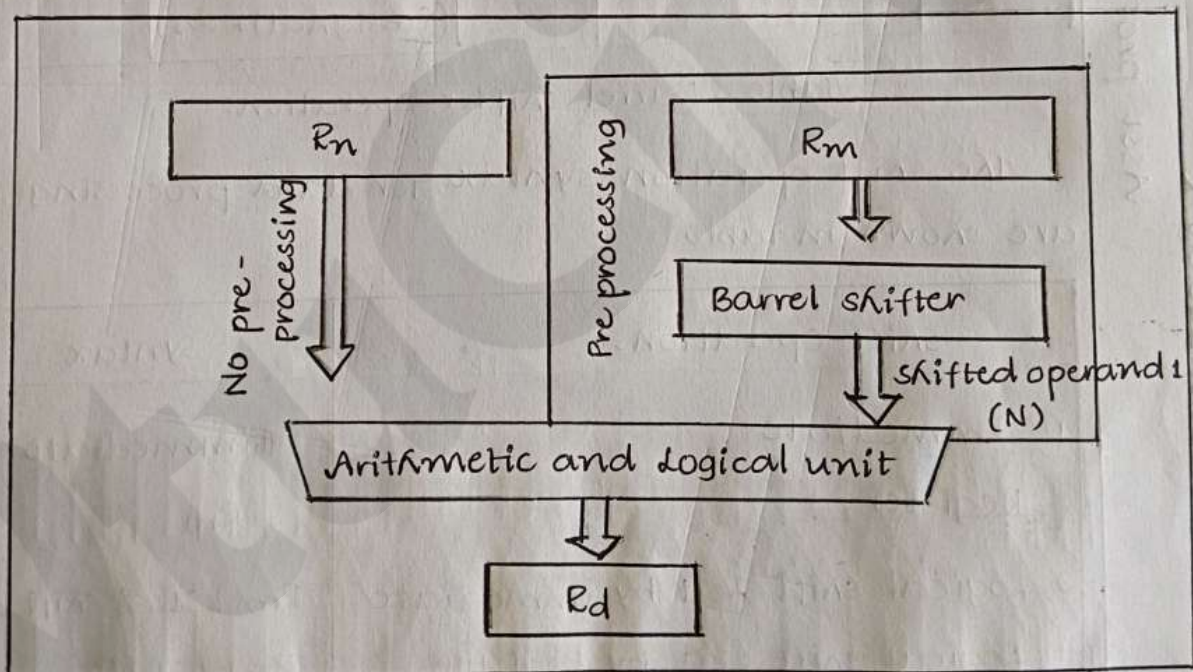
The general structure of the data processing instructions allow for conditional execution, a logical shift of up to 32 bits and the data operation all in the one cycle.

Barrel Shifter:

The general structure of the data processing instructions allows for conditional execution, a logical shift of upto 32 bits and the data operation all in one cycle. The operand 2 can be more than just a register or immediate value; it can also be a register R_m that has been preprocessed by the barrel shifter prior to being by a data processing instruction.

Data processing instructions are processed within the arithmetic logic unit (ALU). A unique and powerful feature of the ARM processor is the ability to shift the 32 bit binary pattern in one of the source registers left or right by a specific number of positions before it enters the ALU. This shift increases the power and flexibility of many data processing operations.

There are data processing instructions that do not use the barrel shift, for example, the MUL (multiply), CLZ (count leading zeroes), and QADD (signed saturated 32-bit add) instructions. Preprocessing or shift occurs within the cycle time of the instruction. This is particularly useful for loading constants into a register and achieving fast multiplies or division by a power of 2.



Barrel shifter and
ALU

Barrel Shifter Operations:

Mnemonic	Description	Shift	Result	Shift amount Y
LSL	Logical shift left	$x \ll Y$		#0-31 or R_s
LSR	Logical shift Right	$x \gg Y$	(unsigned) $x \gg Y$	#1-32 or R_s
ASR	Arithmetic right shift	$x \ggg Y$	(signed) $x \ggg Y$	#1-32 or R_s
ROR	rotate right	$x \ggg Y$	$((\text{unsigned}) x \gg Y) \mid (x \ll (32-Y))$	#1-31 or R_s
RRX	Rotate right extended	$x \ggg Y$	$(\text{Cflag} \ll 31) \mid ((\text{unsigned}) x \gg 1)$	none

Table: Barrel shifter operations

The shift operation syntax for data processing instructions are shown in table.

Shift operations	Syntax
1. Immediate	#immediate
2. Register	R_m
3. Logical shift Left by immediate	$R_m, \text{LSL} \# \text{shift_imm}$
4. Logical shift Left by Register	$R_m, \text{LSL} R_s$
5. Logical shift Right by immediate	$R_m, \text{LSR} \# \text{shift_imm}$
6. Logical shift Right by Register	$R_m, \text{LSR} R_s$
7. Arithmetically shift Right by immediate	$R_m, \text{ASR} \# \text{shift_imm}$

8. Arithmetically shift Right by Register	$R_m, ASR R_s$
9. Rotate Right by immediate	$R_m, ROR \#shift_imm$
10. Rotate Right by Register	$R_m, ROR R_s$
11. Rotate Right with Extend	R_m, RRX

Table: Barrel shift operation syntax for data processing instructions.

1. Immediate Operand:

Immediate values are signified by a leading # symbol. The operand is actually stored in the instruction as an 8-bit value with a 4-bit rotation code. The resultant value is the 8 bit value rotated right 0-30 bits (twice the rotation code amount). Only values that can be represented in this form can be encoded as immediate operand.

Syntax: $\#R[immediate]$

Example - $ADD R_0, R_1, \#256$

Let $R_0 = 0x00000000$

$R_1 = 0x00000011$ ($R_0 = R_1 + 256$)

$Im = 0x000000256$

After executing -

$R_0 = 0x00000267$

2. Register Operand:

The register value is used directly. The C flag is unchanged. Note that this is actually a form of the Register Operand, Logical Shift Left by Immediate option with a 0-bit shift.

Syntax: $\langle R_m \rangle$

Example- ADD R_0, R_1, R_2

Let $R_0 = 0x00000000$

$R_1 = 0x00000001$

$R_2 = 0x00000002$

After execution-

$R_0 = 0x00000003$

3. Logical shift Left by Immediate Operand:

The register value is shifted left by an immediate value in the range 0-31. Note that a shift of zero is identical to the encoding for a register operand with no shift. The C flag will be updated with the last value shifted out of R_m unless the shift count is 0.

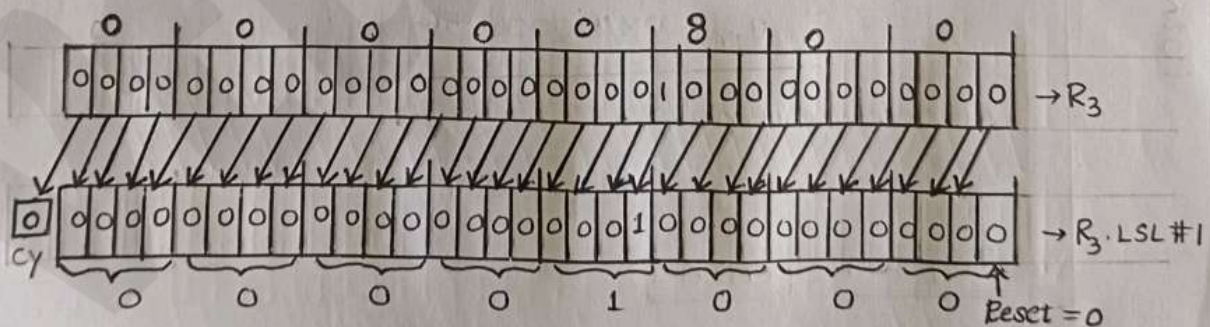
Syntax: $\langle R_m \rangle, LSL \# \langle \text{immediate} \rangle$

Example- ADD $R_0, R_2, R_3, LSL \# 1$

Let $R_0 = 0x00000000$

$R_2 = 0x00000011$

$R_3 = 0x00000800$



$R_2 = 0x00000011$

$R_3.LSL\#1 = 0x00001000$

$R_0 = 0x0000100011$

4. Logical Shift Left by Register Operand :

The register value is shifted left by a value contained in a register. The C flag will be updated with the last value shifted out of R_m unless the value in R_s is 0.

Syntax: $\langle R_m \rangle, LSL \langle R_s \rangle$

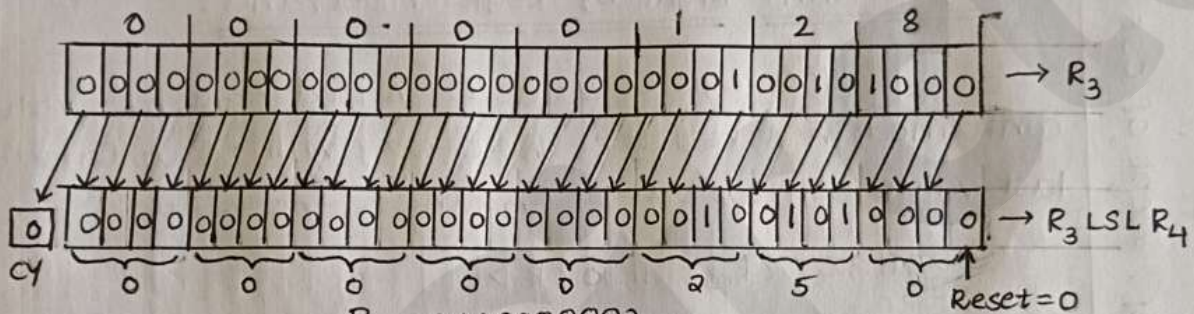
Example- $ADD R_0, R_2, R_3 LSL R_4$

Let $R_0 = 0x00000000$

$R_2 = 0x00000002$

$R_4 = 0x00000001$

$R_3 = 0x00000128$



$R_2 = 0x00000002$

$R_3 = 0x00000250$

$LSL R_4$

$R_0 = 0x00000252$

5. Logical Shift Right by Immediate Operand :

The register value is shifted right by an immediate value in the range 1-32. The C flag will be updated with the last value shifted out of R_m .

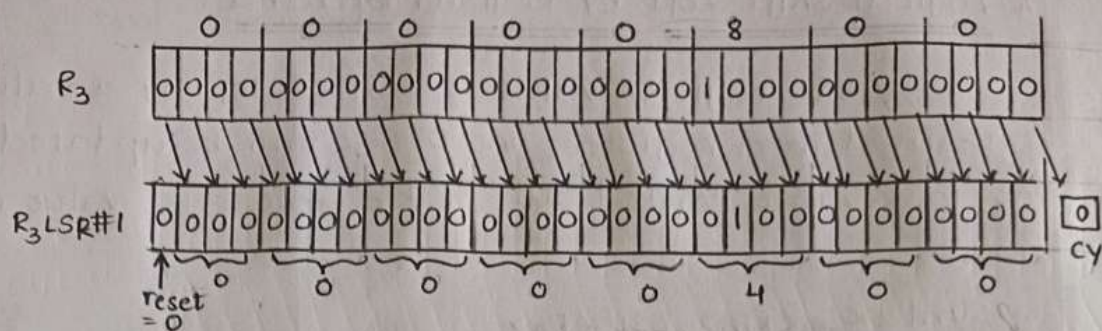
Syntax: $\langle R_m \rangle, LSR \# \langle \text{immediate} \rangle$

Example- $ADD R_0, R_2, R_3 LSR \#1$

Let $R_0 = 0x00000000$

$R_2 = 0x00000017$

$R_3 = 0x00000800$



$$R_2 = 0x00000017$$

$$R_3 \text{ LSR } \#1 = 0x000000400$$

$$R_0 = 0x000000417$$

6. Logical Shift Right by Register Operand:

The register value is shifted right by a value contained in a register. The C flag will be updated with the last value shifted out of R_m unless the value in R_s is 0.

Syntax: $\langle R_m \rangle, \text{LSR } \langle R_s \rangle$

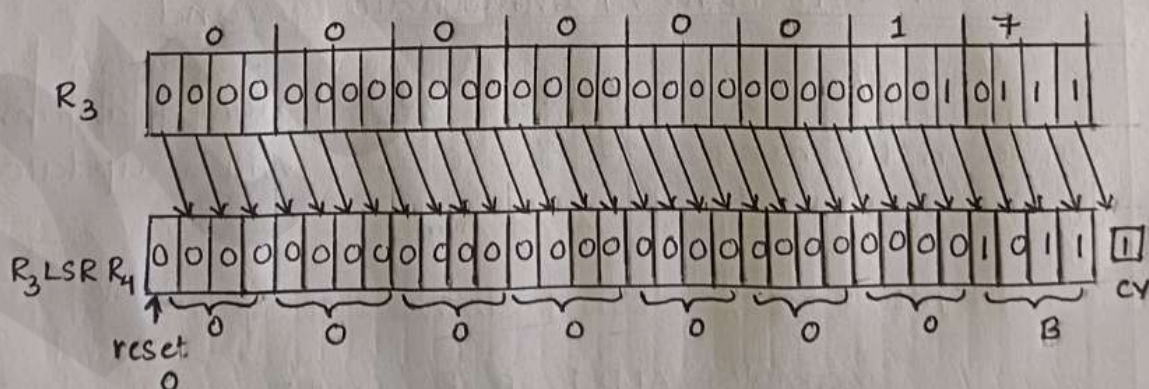
Example - $\text{ADD } R_0, R_2, R_3 \text{ LSR } R_4$

$$\text{Let } R_0 = 0x00000000$$

$$R_2 = 0x00000003$$

$$R_4 = 0x00000001$$

$$R_3 = 0x00000017$$



$$R_2 = 0x00000003$$

$$R_3 \text{ LSR } R_4 = 0x0000000B$$

$$R_0 = 0x0000000E$$

7. Arithmetic shift Right by Immediate Operand:

The register value is arithmetically shifted right by an immediate value in the range 1-32. The arithmetic shift fills from the left with the sign bit, preserving the sign of the number. The C flag will be updated with the last value shifted out of R_m .

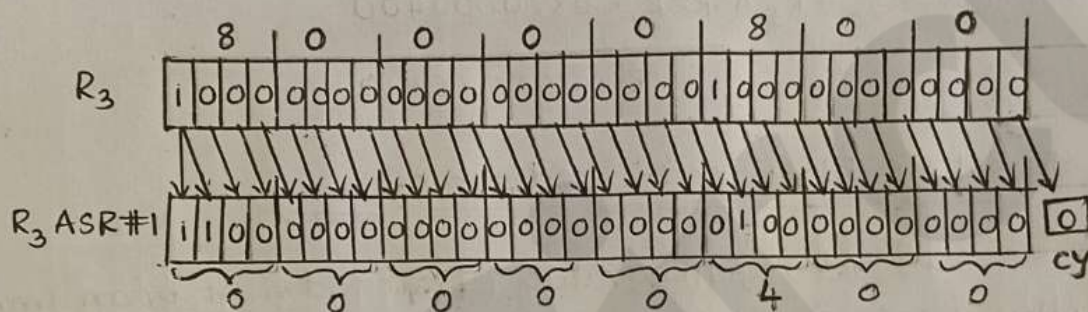
Syntax: $\langle R_m \rangle, ASR \# \langle \text{Immediate} \rangle$

Example- $ADD R_0, R_2, R_3, ASR \#1$

Let $R_0 = 0x00000000$

$R_2 = 0x00000011$

$R_3 = 0x80000800$



$R_2 = 0x00000011$

$R_3 ASR \#1 = 0xC0000400$

$R_0 = 0xC0000411$

8. Arithmetic Shift Right by Register Operand:

The register value is arithmetically shifted right by a value contained in a register. The arithmetic shift fills from the left with the sign bit, preserving the sign of the number. The C flag will be updated with the last value shifted out of R_m unless the value of R_s is 0.

Syntax: $\langle R_m \rangle ASR \langle R_s \rangle$

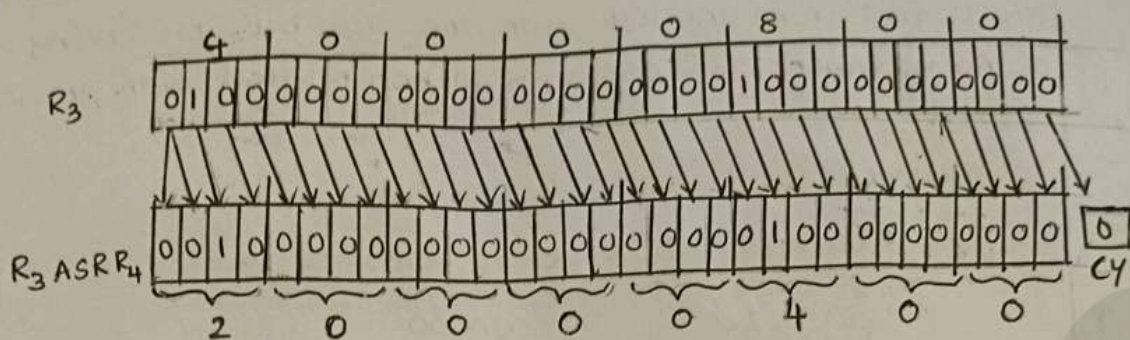
Example- $ADD R_0, R_2, R_3, ASR R_4$

Let $R_0 = 0x00000000$

$R_2 = 0x00000002$

$R_4 = 0x00000001$

$R_3 = 0x40000800$



$R_2 = 0x00000002$

$R_3 \text{ ASR } R_4 = 0x20000400$

$R_0 = 0x20000402$

9. Rotate Right by Immediate Operand:

The register value is rotated right by an immediate value in the range 1-31. A rotate value of 0 in this instruction encoding will cause an RRX operation to be performed. The C flag will be updated with the last value shifted out of R_m .

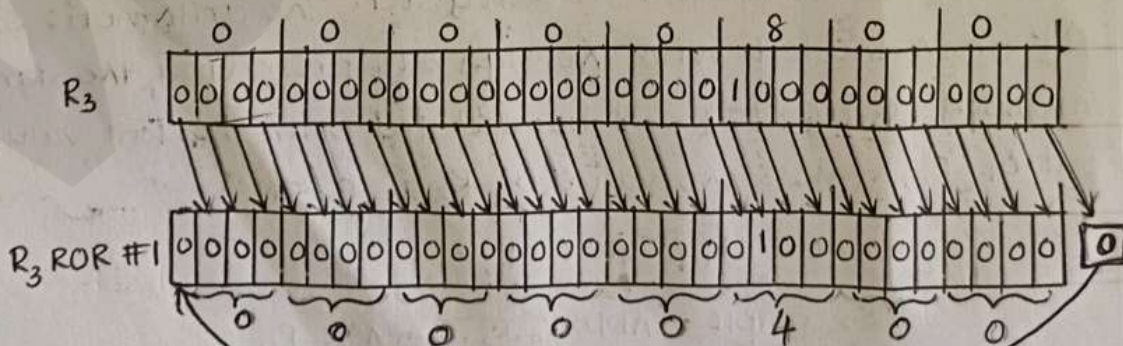
Syntax: $\langle R_m \rangle, \text{ROR} \# \langle \text{immediate} \rangle$

Example- $\text{ADD } R_0, R_2, R_3, \text{ROR} \# 1$

Let $R_0 = 0x00000000$

$R_2 = 0x00000002$

$R_3 = 0x00000800$



$$R_2 = 0x00000002$$

$$R_3 \text{ ROR } \#1 = 0x00000400$$

$$R_0 = 0x00000402$$

10. Rotate Right by Register Operand:

The register value is rotated right by a value contained in a register. The C flag will be updated with the last value shifted out of R_m unless the value in R_3 is 0.

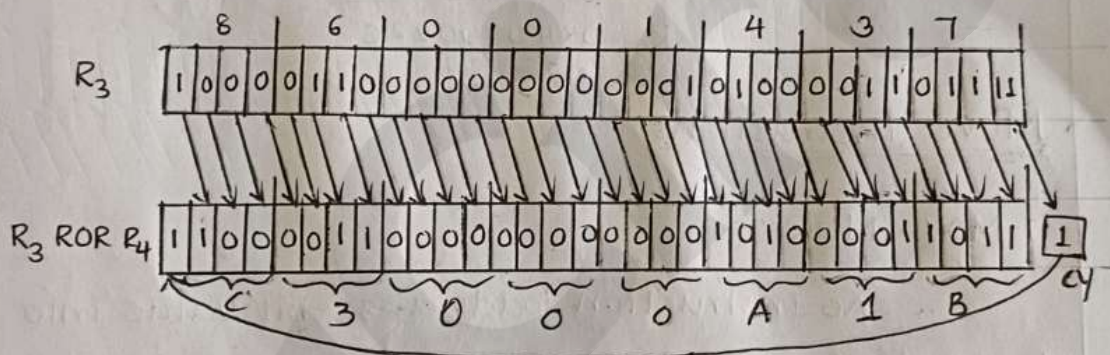
Syntax: $\langle R_m \rangle, \text{ROR } \langle R_s \rangle$

Example - $\text{ADD } R_6, R_2, R_3 \text{ ROR } R_4$

$$\text{Let } R_6 = 0x00000000$$

$$R_2 = 0x00000011 \quad R_4 = 0x00000001$$

$$R_3 = 0x86001437$$



$$R_2 = 0x0000000011$$

$$R_3 \text{ ROR } R_4 = 0xC3000A1B$$

$$R_0 = 0xC3000A1B2C$$

11. Rotate Right with Extend Operand:

The register value is rotated right with one bit through the C flag, i.e., $C \leftarrow R_m[0]$, $R_m[31] \leftarrow C$, $R_m[30] \leftarrow R_m[29]$, etc.

Syntax: $\langle R_m \rangle, \text{RRX}$

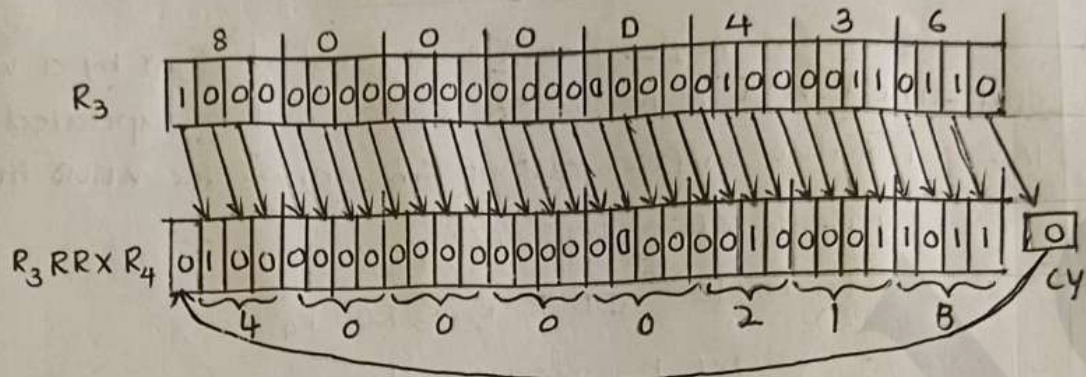
Example - ADD R_0, R_2, R_3 RRX R_4

Let $R_0 = 0x00000000$

$R_2 = 0x00000011$

$R_4 = 0x00000001$

$R_3 = 0x8000436$



$R_2 = 0x00000011$

$R_3 \text{ RRX } R_4 = 0x4000021B$

$R_0 = 0x4000022C$

MOVE instructions:

1. MOV - Move:

The instruction loads a 32-bit value into the destination register, from another register, a shifted register, or an immediate value.

Syntax: $\text{MOV} \{ \langle \text{cond} \rangle \} \{ s \} \langle R_d \rangle, \langle \text{shifter_operand} \rangle$

(or)

$\text{MOV} \langle R_d \rangle, \langle \text{shifter_operand} \rangle$

MOV performs a move to a register from another register or an immediate value.

$\text{MOV } R_1, R_0$; $R_1 \leftarrow R_0$

$\text{MOV } R_1, R_0, \text{LSL} \#2$; $R_1 \leftarrow R_0 * 4$

$\text{MOV } R_1, \#1$; $R_1 \leftarrow 0x00000001$

Example-

MOV R_1, R_0 ; $R_1 \leftarrow R_0$

This instruction will move (copy) the content of operand two R_0 into operand one R_1 , without changing the content of operand two

Let

$R_1 = 0x00000000$

$R_0 = 0x00000011$

After execution of this instruction

$R_1 = 0x00000011$

$R_0 = 0x00000011$

2. MVN - Move Negative:

Syntax: MVN $\langle R_d \rangle, \langle \text{shifter_operand} \rangle$

This instruction loads a value into the destination register, from another register, a shifted register, or an immediate value. The difference is the bits are inverted prior to moving, thus you can move a negative value into a register. Due to the way this works (two's complement), you want to move one less than the required number.

MVN R_1, R_0 ; $R_1 \leftarrow \text{NOT } R_0$

MVN $R_1, R_0, \text{LSL } \#2$; $R_1 \leftarrow \text{NOT}(R_0 \ll 2)$

MVN $R_0, \#4$; $R_0 = -5$

Example-

MVN R_1, R_0 ; $R_1 \leftarrow \text{NOT } R_0$

This instruction will move the complimented value of the content of operand two (R_0) into operand one (R_1), without changing the content of operand two.

Let $R_1 = 0x00000000$

$R_0 = 0x00000008$

After the execution of the instruction

$R_1 = 0xFFFFFFFF$ (two's complement of 1) = -1

$R_0 = 0x00000008$

Arithmetic Instructions:

Mnemonic	Meaning
1. ADD	Addition
2. ADC	Addition with carry
3. SUB	Subtract
4. RSB	Reverse Subtract
5. SBC	Subtract with carry
6. RSC	Reverse Subtract with carry.

1. ADD - Addition:

Syntax: ADD $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will add the two operands, placing the result in the destination register. Operand 1 is a register, operand 2 can be a register, shifted register, or an immediate value.

ADD R_0, R_1, R_2 ; $R_0 = R_1 + R_2$

ADD $R_0, R_1, \#256$; $R_0 = R_1 + 256$

ADD $R_0, R_2, R_3, \text{LSL}\#1$; $R_0 = R_2 + (R_3 \ll 1)$

The addition may be performed on signed or unsigned numbers.

Example - ADD R_0, R_1, R_2 ; $R_0 = R_1 + R_2$

Here operand 2 is a register R_2

$$\text{Let } R_0 = 0x00000000$$

$$R_1 = 0x00000011$$

$$R_2 = 0x00000022 \quad (\because R_0 = R_1 + R_2)$$

$$R_0 = 0x00000033$$

2. ADC - Addition with carry :

Syntax: $\text{ADC } \langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will add the two operands, placing the result in the destination register. It uses a carry bit, so can add numbers larger than 32 bits. Operand 1 is a register, operand 2 can be a register, shifted register or an immediate value.

Example-

$\text{ADDS } R_4, R_2, R_0$; add least significant words

$\text{ADC } R_5, R_3, R_1$; add most significant words plus carry.

Let

$$R_5 = 0x00000000 \quad \left. \begin{array}{l} R_5 = 0x00000000 \\ R_4 = 0x00000000 \end{array} \right\} \rightarrow \text{Destination registers}$$

$$R_4 = 0x00000000$$

$$R_3 = 0x00000800 \rightarrow \text{MSB word}$$

$$R_2 = 0x00000310 \rightarrow \text{LSB word}$$

$$R_1 = 0x00000300 \rightarrow \text{MSB word}$$

$$R_0 = 0x00000250 \rightarrow \text{LSB word}$$

$\left. \begin{array}{l} R_3 = 0x00000800 \rightarrow \text{MSB word} \\ R_2 = 0x00000310 \rightarrow \text{LSB word} \end{array} \right\} \rightarrow 64 \text{ bit data 1}$

$\left. \begin{array}{l} R_1 = 0x00000300 \rightarrow \text{MSB word} \\ R_0 = 0x00000250 \rightarrow \text{LSB word} \end{array} \right\} \rightarrow 64 \text{ bit data 2}$

Now, After $\text{ADDS } R_4, R_2, R_0$

$$R_0 = 0x00000250$$

$$R_2 = 0x00000310$$

$$R_4 = 0x00000560$$

$$(\because R_4 = R_0 + R_2)$$

with carry = 0

After ADC R_5, R_3, R_1

$$R_3 = 0x00000800 \quad (\because R_5 = R_3 + R_1 + \text{carry})$$

$$R_1 = 0x00000300$$

$$= 0x00000B00 + \text{carry}(0)$$

$$R_5 = 0x00000B00$$

3. SUB - Subtraction:

Syntax: SUB $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will subtract operand two from operand one, placing the result in the destination register. Operand 1 is a register, operand 2 can be a register, shifted register, or an immediate value.

The subtraction may be performed on signed or unsigned numbers.

$$\text{SUB } R_0, R_1, R_2 \quad ; R_0 = R_1 - R_2$$

$$\text{SUB } R_0, R_1, \#0x00008000 \quad ; R_0 = R_1 - 0x00008000$$

$$\text{SUB } R_0, R_2, R_3, \text{LSL}\#1 \quad ; R_0 = R_2 - (R_3 \ll 1)$$

Example - 1

$$\text{SUB } R_0, R_1, R_2$$

Here operand 2 is a register R_2

$$\text{Let } R_0 = 0x00000000$$

$$R_1 = 0x00000033$$

$$R_2 = 0x00000011$$

$$\text{Now, } R_0 = R_1 - R_2$$

$$R_1 = 0x00000033$$

$$R_2 = 0x00000011$$

$$R_0 = 0x00000022$$

Example-2

SUB $R_0, R_1, \#0x00008000$;

Here operand 2 is an immediate data $0x00008000$

Let $R_0 = 0x00000000$

$R_1 = 0x0000FFFF$

Immediate data = $0x00008000$

$$R_0 = R_1 - 0x00008000$$

$$R_1 = 0x0000FFFF$$

$$\text{Imm} = 0x00008000$$

$$R_0 = 0x00007FFF$$

4. SBC: Subtraction with carry -

Syntax: SBC $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will subtract the two operands, placing the result in the destination register. It uses the carry bit to represent 'borrow', so can subtract numbers larger than 32 bits. SUB and SBC generate the carry flag the wrong way around, if a borrow is required then the carry flag is UNSET. Thus, this instruction requires a NOT Carry flag - it inverts the flag automatically during the instruction. The subtraction may be performed on signed or unsigned numbers.

SBC R_0, R_1, R_2 ; $R_0 = R_1 - R_2 - \text{NOTC}$

SBC $R_0, R_1, \#0x00008000$; $R_0 = R_1 - 0x00008000 - \text{NOTC}$

SBC $R_0, R_2, R_3, \text{LSL}\#1$; $R_0 = R_2 - (R_3 \ll 1) - \text{NOTC}$

Example-1

SBC R_0, R_1, R_2

Here operand 2 is a Register R_2

Let $R_0 = 0x00000000$

$R_1 = 0x00000033$

$$R_2 = 0x00000011$$

$$C=0 \Rightarrow NOTC=1$$

$$\text{Now } R_0 = R_1 - R_2 - NOTC$$

$$R_1 = 0x00000033$$

$$R_2 = 0x00000011$$

$$R_0 = 0x00000022 - 1$$

$$R_0 = 0x00000021$$

Example-2

$$SBC R_0, R_1, \#0x00008000$$

Here operand 2 is an immediate data 0x00008000

$$\text{Let } R_0 = 0x00000000$$

$$R_1 = 0x0000FFFF$$

$$C=1 \Rightarrow NOTC=0$$

$$\text{Immediate Data} = 0x00008000$$

$$\text{Now, } R_0 = R_1 - 0x00008000 - NOTC$$

$$R_1 = 0x0000FFFF$$

$$\text{Imm} = 0x00008000$$

$$R_0 = 0x0000FFFF - 0$$

$$R_0 = 0x0000FFFF$$

5. RSB : Reverse Subtract -

Syntax: $RSB \langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter operand} \rangle$

This instruction will subtract operand one from operand two, placing the result in the destination register. Please note that normal subtraction operand 2 is subtracted from operand 1. Operand 1 is a register, operand 2 can be a register, shifted register, or an immediate value. Normally this instruction is used for "negating or 2's Complementing" of numbers.

RSB R_0, R_1, R_2 ; $R_0 = R_2 - R_1$
 RSB $R_0, R_1, \#256$; $R_0 = 256 - R_1$
 RSB $R_0, R_2, R_3, LSL\#1$; $R_0 = (R_3 \ll 1) - R_2$

The subtraction may be performed on signed or unsigned numbers.

Example - RSB R_0, R_1, R_2

Here operand 2 is a register R_2

Let $R_0 = 0x00000000$

$R_1 = 0x00000033$

$R_2 = 0x00000186$

Now $R_0 = R_2 - R_1$

$R_2 = 0x00000186$

$R_1 = 0x00000033$

$R_0 = 0x00000153$

6. RSC: Reverse subtract with carry -

Syntax: RSC $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will subtract "Operand one and complemented value of carry from operand two" placing the result in the destination register.

RSC R_0, R_1, R_2 ; $R_0 = R_2 - R_1 - \text{NOTC}$

RSC $R_0, R_1, \#256$; $R_0 = 256 - R_1 - \text{NOTC}$

RSC $R_0, R_2, R_3, LSL\#1$; $R_0 = (R_3 \ll 1) - R_2 - \text{NOTC}$

The subtraction may be performed on signed or unsigned numbers

Example-1

RSC R_0, R_1, R_2

Here operand 2 is a register R_2

$$\text{Let } R_0 = 0x00000000$$

$$R_2 = 0x00000186$$

$$R_1 = 0x00000121$$

$$C = 0$$

$$\text{Now } R_0 = R_2 - R_1 - \text{NOTC}$$

$$R_2 = 0x00000186$$

$$R_1 = 0x00000121$$

$$R_0 = 0x00000065$$

$$-1$$

$$R_0 = 0x00000064$$

Example-2

$$\text{RSC } R_0, R_1, \#0x0000FFFF$$

$$\text{Let } R_0 = 0x00000000, R_1 = 0x00008000$$

$$C = 1$$

$$\text{Now, } R_0 = \#imm - R_1 - \text{NOTC}$$

$$\#imm = 0x0000FFFF$$

$$R_1 = 0x00008000$$

$$R_0 = 0x00007FFF$$

$$-0$$

$$R_0 = 0x00007FFF$$

Logical Instructions:

1. AND: Logical AND -

Syntax: AND $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will perform a logical AND operation between the two operands, placing the result in the destination register; this is useful for masking the bits you wish to work on. Operand 1 is a register; operand 2 can be a register, shifted register, or an immediate value:

AND R_0, R_1, R_2 ; $R_0 = R_1 \text{ AND } R_2$ AND $R_0, R_1, \#0x00008000$; $R_0 = R_1 \text{ AND } 0x00008000$ AND $R_0, R_2, R_3, \text{LSL} \#1$; $R_0 = R_2 \text{ AND } (R_3 \ll 1)$

Example-1

AND R_0, R_1, R_2 Let $R_0 = 0x00000000$ $R_1 = 0x00000011$ $R_2 = 0x00000033$ $R_1 = 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0001 \ 0001$ $R_2 = 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0011 \ 0011$ $R_0 = 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0001 \ 0001$ $\therefore R_0 = 0x00000011$

Example-2

AND $R_0, R_1, \#0x00008000$ Let $R_1 = 0x0FFFFFF0$ $R_1 = 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 0000 \ 0111 \ 0000$ $\# = 0000 \ 0000 \ 0000 \ 1000 \ 1000 \ 1000 \ 0000 \ 0000$ $R_0 = 0000 \ 0000 \ 0000 \ 0000 \ 1000 \ 0000 \ 0000 \ 0000$ $\therefore R_0 = 0x00008000$ 2. ORR: Logical OR-Syntax: ORR $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will perform a logical OR between the two operands, placing the result in the destination register; this is useful for setting certain bits to be set. operand 1 is a register, operand 2 can be a register, shifted register, or an immediate value:

ORR R_0, R_1, R_2 ; $R_0 = R_1 \text{ OR } R_2$
 ORR $R_0, R_1, \#0x80000000$; $R_0 = R_1 \text{ OR } 0x80000000$
 ORR $R_0, R_2, R_3, \text{LSL}\#1$; $R_0 = R_2 \text{ OR } (R_3 \ll 1)$

Example-1

ORR R_0, R_1, R_2

Let $R_1 = 0x00000066$

$R_0 = 0x00000000$

$R_2 = 0x01201267$

Now $R_0 = R_1 \text{ OR } R_2$

$R_1 = 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0110 \ 0110$

$R_2 = 0000 \ 0001 \ 0010 \ 0000 \ 0001 \ 0010 \ 0110 \ 0111$

$R_0 = \underbrace{0000}_0 \ \underbrace{0001}_1 \ \underbrace{0010}_2 \ \underbrace{0000}_0 \ \underbrace{0001}_1 \ \underbrace{0010}_2 \ \underbrace{0110}_6 \ \underbrace{0111}_7$

$R_0 = 0x01201267$

Example-2

ORR $R_0, R_1, \#0x80000000$

Here operand 2 is an immediate data $0x80000000$

Let $R_0 = 0x00000000$

$R_1 = 0x00000111$

Immediate Data = $0x80000000$

Now, $R_0 = R_1 \text{ OR } 0x80000000$

$R_1 = 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0001 \ 0001 \ 0001$

Imm = $1000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$

$R_0 = \underbrace{1000}_8 \ \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0001}_1 \ \underbrace{0001}_1 \ \underbrace{0001}_1$

$R_0 = 0x80000111$

3. EOR : Logical Exclusive-OR -

Syntax: EOR $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction will perform a logical Exclusive OR between the two operands, placing the result in the destination register: this is useful for inverting certain bits. Operand 1 is a register, operand 2 can be a register, shifted register, or an immediate value.

EOR R_0, R_1, R_2 ; $R_0 = R_1 \text{ X-OR } R_2$
EOR $R_0, R_1, \#0x80000000$; $R_0 = R_1 \text{ X-OR } 0x80000000$
EOR $R_0, R_2, R_3, \text{LSL}\#1$; $R_0 = R_2 \text{ X-OR } (R_3 \ll 1)$

Example: 1-

EOR R_0, R_1, R_2

Let $R_1 = 0x00000066$

$R_2 = 0x01201267$

$R_1 =$ 0000 0000 0000 0000 0000 0000 0110 0110

$R_2 =$ 0000 0001 0010 0000 0001 0010 0110 0111

$R_0 =$ 0000 0001 0010 0000 0001 0010 0000 0001
 0 1 2 0 1 2 0 1

$R_0 = 0x01201201$

4. BIC : Bit clear-

Syntax: BIC $\langle R_d \rangle, \langle R_n \rangle, \langle \text{shifter_operand} \rangle$

BIC is a way to clear bits within a word, a sort of reverse OR. Operand two is a 32 bit mask. If a bit is set in the mask, it will be cleared. Unset mask bits indicate bits to be left alone. This instruction is particularly useful when clearing status bits and is frequently used to change interrupt masks in the CPSR. This instruction performs the

logical AND operation between operand 1 and complemented value of operand 2.

operand 1 is a register, operand 2 can be a register, shifted register, or an immediate value:

BIC R_0, R_1, R_2 ; $R_0 = R_1 \& \sim R_2$

BIC $R_0, R_1, \#256$; $R_0 = R_1 \& \sim 0X00000256$

BIC $R_0, R_1, R_2, LSL\#1$; $R_0 = R_1 \& \sim (R_2 \ll 1)$

Example - 1

BIC R_0, R_1, R_2

Here operand 2 is a register R_2

Let $R_0 = 0X00000000$

$R_1 = 0X0000000F$

$R_2 = 0X00000005$

Now $R_1 \& \sim R_2$

$R_1 =$	0000	0000	0000	0000	0000	0000	0000	1111
$\sim R_2 =$	1111	1111	1111	1111	1111	1111	1111	1010
$R_0 =$	<u>0000</u>	<u>0000</u>	<u>0000</u>	<u>0000</u>	<u>0000</u>	<u>0000</u>	<u>0000</u>	<u>1010</u>
	0	0	0	0	0	0	0	A

$R_0 = 0X0000000A$

Example - 2

BIC $R_0, R_1, \#0X00000005$

Here operand 2 is an immediate value $0X00000005$

Let $R_0 = 0X00000000$

$R_1 = 0X00000005$

Immediate Data = $0X00000011$

Now $R_0 = R_1 \& \sim \text{Immediate Data}$

$$R_1 = 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0001 \ 0001$$

$$\sim R_2 = 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1010$$

$$R_0 = \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0000}_0 \ \underbrace{0001}_1 \ \underbrace{0000}_0$$

$$R_0 = 0x00000010$$

Comparison Instructions:

The comparison instructions are used to compare or test a register with a 32-bit value. They update the cpsr flag bits according to the result, but do not affect other registers.

1. CMP: Compare-

Syntax: $CMP \langle R_n \rangle, \langle shifter_operand \rangle$

The CMP instruction performs a subtraction, but does not store the result. The flags are always updated. CMP always allow you to compare the contents of a register with another register or an immediate value, updating the status flags to allow conditional execution to take place.

Example -

$CMP \ R_0, \#1$; $Z=1$ if $R_0=1$, $N=0$ if $R_0>1$

$CMP \ R_0, R_1$; $Z=1$ if $R_0=R_1$, $N=0$ if $R_0>R_1$

$CMP \ R_0, R_1, LSL \#2$; $Z=1$ if $R_0 = (R_1, LSL \#2)$,
 $N=0$ if $R_0 > (R_1, LSL \#2)$

2. TST: Test-

Syntax: $TST \langle R_n \rangle, \langle shifter_operand \rangle$

It performs logical AND operation between operand 1 and operand 2 without storing the result, but only the flags are always updated.

TST R_0, R_1 ; $z=1$ if $R_0=R_1$, otherwise reset to 0
 TST $R_0, \#0x00008000$; $z=1$ if 15th bit of $R_0=0$
 otherwise reset to 0.

3. TEQ : Test Equivalence -

Syntax: TEQ $\langle R_n \rangle, \langle \text{shifter_operand} \rangle$

This instruction performs XOR operation between operand 1 and operand 2 without storing the result, but only the flags are always updated.

Example -

TEQ $R_0, \#0x00008000$; sets $z=1$ if R_0 contains value $0x00008000$

TEQ R_0, R_1 ; sets $N=1$ if signs are different.

consider TEQ $R_0, \#0x80008000$

Let $R_0 = 0x00000033$

Then R_0 value is XOR with immediate data $0x00008000$

$R_0 =$	0000	0000	0000	0000	0000	0000	0011	0011
Data =	1000	0000	0000	0000	1000	0000	0000	0000
Result =	1000	0000	0000	0000	1000	0000	0011	0011

Since the result is not zero

$Z=0, N=1$

Multiply Instructions:

The multiply instructions multiply the contents of a pair of registers and depending upon the instruction, accumulate the results in with another register. The long multiplies accumulate onto a pair of registers representing a 64-bit value. The final result is placed in a destination register or a pair of registers.

In addition to the barrel shifter, the ARM7 has a built-in Multiply Accumulate unit (MAC). The MAC supports integer and long integer multiplication.

Mnemonic	Meaning	Resolution
1. MUL	Multiply	32 bit result
2. MLA	Multiply and accumulate	32 bit result

These two instructions are different from normal arithmetical instructions in that there are restrictions on the operands, namely:

1. All operands, and the destination, must be given as simple registers.
2. You cannot use immediate values or shifted registers for operand two.
3. The destination and operand one must be different registers.
4. Lastly, you cannot specify R15 as the destination.

1. MUL: Multiply -

Syntax: MUL $\langle R_d \rangle, \langle R_m \rangle, \langle R_s \rangle$

This instruction performs a 32×32 multiply operation and stores 32-bit result.

Example -

MUL R_2, R_1, R_0

$R_0 = 0x00000002$

$R_1 = 0x00000004$

$R_2 = 0x00000000$

After the execution of this instruction

$$R_2 = R_1 \times R_0$$

$$= (0 \times 00000004) \times (0 \times 00000002)$$

$$R_2 = 0 \times 00000008$$

2. MLA : Multiply and Accumulate -

MLA behaved that same as MUL, except that the value of operand three is added to the result.

Syntax: $MLA \langle R_d \rangle, \langle R_m \rangle, \langle R_s \rangle, \langle R_n \rangle$

Example -

$MLA \ R_3, R_2, R_1, R_0$

This instruction below adds the product of R_1 and R_2 to R_0 and stores the result in R_3

Let $R_0 = 0 \times 00000004$

$$R_1 = 0 \times 00000004$$

$$R_2 = 0 \times 00000009$$

$$R_3 = 0 \times 00000000$$

After the execution of instruction

$$R_3 = (R_1 \times R_2) + R_0$$

$$= (0 \times 00000004 \times 0 \times 00000009) + 0 \times 00000004$$

$$R_3 = 0 \times 0000000C$$

MAC unit:

In addition to the barrel shifter, the ARM1 has a built-in Multiply Accumulate Unit (MAC). The MAC supports integer and long integer Multiplication. The integer multiplication instructions support multiplication of two 32-bit registers and place the result in a third 32-bit register. A multiply-accumulate instruction will take

the same product and add it to a running total. Long integer multiplication allows two 32-bit quantities to be multiplied together and the 64-bit result is placed in two registers. Similarly a long multiply and accumulate is also available.

Mnemonic	Meaning	Resolution
MUL	Multiply	32 bit result
MLA	Multiply accumulate	32 bit result
UMULL	unsigned multiply	64 bit result
UMLAL	unsigned multiply accumulate	64 bit result
SMULL	signed multiply	64 bit result
SMLAL	signed multiply accumulate	64 bit result

Branch Instructions :

A branch instruction changes the flow of execution or is used to call a routine. This type of instruction allows programs to have subroutines, if-then-else structures, and loops.

The ARMv5E instruction set includes four different branch instructions.

CSE, CIT/NC

① Branch (B)

② Branch with link (BL)

③ Branch Exchange (BX)

④ Branch Exchange with link (BLX)

Syntax :

B { <Cond> } label

BL { <Cond> } label

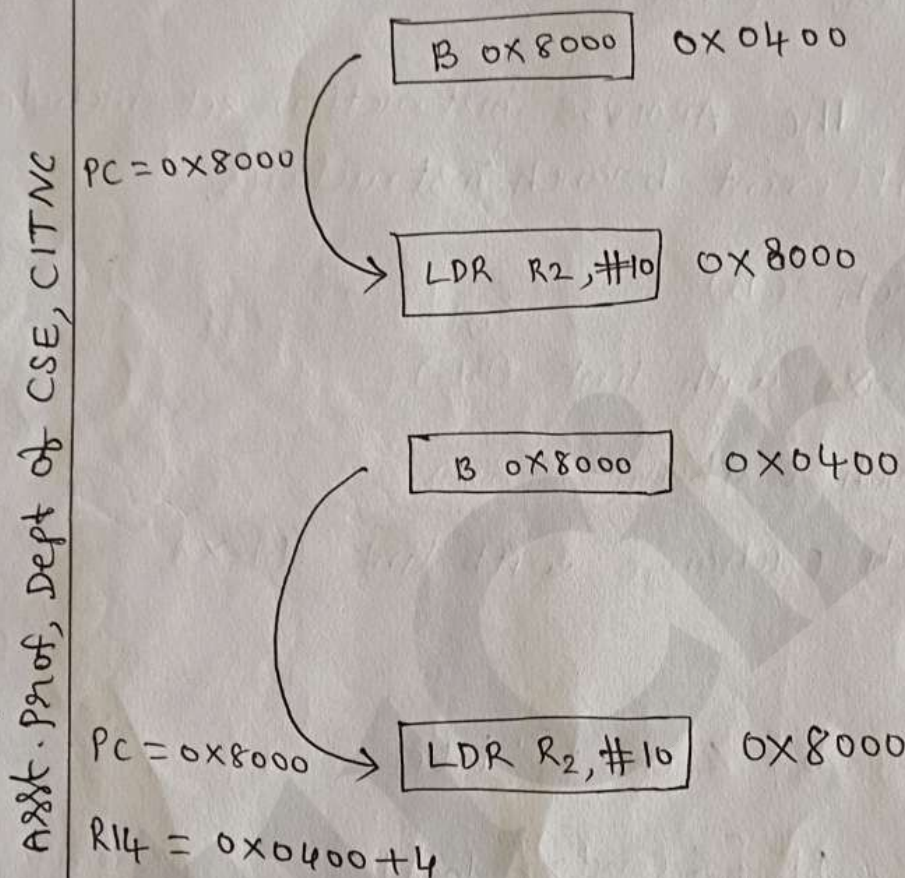
BX { <Cond> } Rm

BLX { <Cond> } label Rm

① Branch (B) :

The basic branch instruction allows a jump forward or backwards of upto 32 MB.

A modified version of the branch instruction, the branch link, allows the same jump but stores the current PC address plus four bytes in the link Register (LR).



The branch instruction will jump you to a destination address. The branch link instruction jumps to the destination and stores a return address in R14.

The branch instructions have two other variants called "branch exchange" and "branch exchange with link". These two instructions perform the same branch operation but also swap instruction operation from ARM to THUMB and vice versa.

Example:

This example shows a forward and backward branch. Because these loops are address specific, we do not include the pre and post conditions. The forward branch skips three instructions. The backward branch creates an infinite loop.

B forward

ADD $r_1, r_2, \#4$

ADD $r_0, r_6, \#2$

ADD $r_3, r_7, \#4$

forward:

SUB $r_1, r_2, \#4$

Backward:

ADD $r_1, r_2, \#4$

SUB $r_1, r_2, \#4$

ADD r_4, r_6, r_7

B Backward

In this example, forward and backward are the labels. The branch labels are placed at the beginning of the line and are used to mark an address that can be used later by the assembler to calculate the branch offset.

② Branch with Link (BL):

The Branch with link (BL) instruction is similar to the Branch (B) instruction, but overwrites the link register (LR) with a return address. It performs a subroutine call.

Example:

This example shows a simple fragment of code that branches to a subroutine using the BL instruction. To return from a subroutine, you copy the link register to the PC.

```
BL    Subroutine    ; branch to subroutine
CMP   r1, #5        ; Compare r1 with 5
MOVEQ r1, #0        ; if (r1 == 5) then r1 = 0
```

Subroutine:

< Subroutine Code >

```
MOV   PC, LR        ; return by moving
                     PC = LR
```

ex:-

```
BL    ADDITION
CMP   R2, #8
MOVEQ R2, #0
```

ADDITION:

```
ADD   R1, R2, #4
SUB   R1, R2, #4
ADD   R4, R6, R7
MOV   PC, LR.
```

Software Interrupt Instructions: (SWI)

The Software Interrupt Instruction generates an exception on execution, forces the processor into Supervisor mode and jumps the PC to 0x0000 0008.

The Software Interrupt Instruction forces the CPU into SUPERVISOR mode and jumps the PC to the SWI vector table offset (0x8).

Each SWI instruction has an associated SWI number, which is used to represent a particular function call or feature.

The SWI number is determined by

$$\text{SWI_Number} = \langle \text{SWI instruction} \rangle \text{ AND NOT } (0xff000000).$$

Syntax:

$\text{SWI } \{ \langle \text{cond} \rangle \} \langle \text{immediate}_{24} \rangle$

If (Cond)

$R_{14_svc} \leftarrow$ address of next instruction
after SWI instruction

$\text{SPSR}_{svc} \leftarrow \text{CPSR}$; save current
CPSR

$\text{CPSR}[4:0] \leftarrow 10011b$; Supervisor mode

$\text{CPSR}[5] \leftarrow 0$; ARM execution

$\text{CPSR}[7] \leftarrow 1$; disable interrupts

$\text{PC} \leftarrow 0x00000008$; jump to exception
vector.

The SWI instruction causes a SWI exception. The processor disables interrupts, switches to ARM execution, if previously in Thumb and enters Supervisory mode.

Example:-

Here we have a simple example of an SWI call with SWI number 0x123456, used by ARM toolkits as a debugging SWI. Typically the SWI instruction is executed in user mode.

PRE

$\text{CPSR} = \text{nzcvqif t} - \text{USER}$

$\text{PC} = 0x00008000$

Pothi Reddy.K

Asst. Prof, Dept of CSE, CITNC

LR = 0x003ffffff ; LR = R₁₄

R₀ = 0x12

0x00008000 SWI 0x123456

Post

CPSR = nzcvqift_SVC

SPSR = nzcvqift_USER

PC = 0x000b0008

LR = 0x00008004

R₀ = 0x12

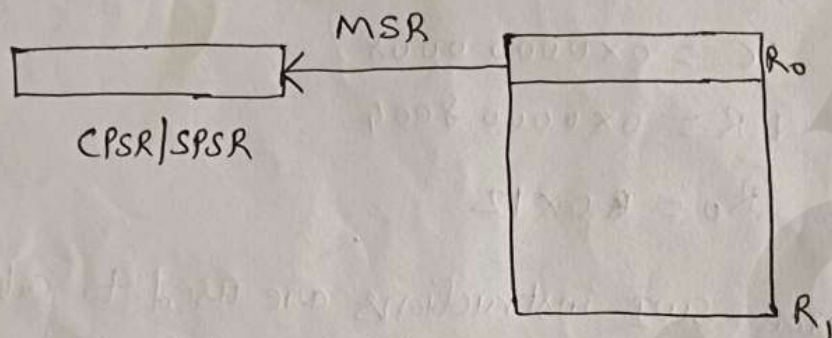
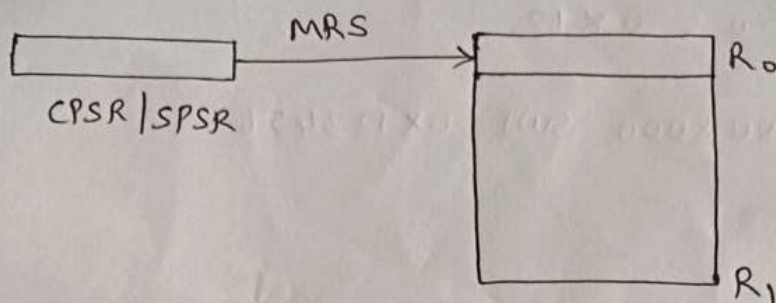
Since SWI instructions are used to call operating system routines, you need some form of parameter passing. This is achieved using registers. In this example, register "R₀" is used to pass the parameter 0x12. The return values are also passed back via registers.

Program Status Register Instructions : (PSR)

The ARM instruction set provides two instructions to directly control a program status register (PSR).

(i) The MRS instruction transfers the contents of either the CPSR or SPSR into a register

(ii) The MSR instruction transfers the contents of a register into the CPSR or SPSR.



Syntax:

$MRS \{ \langle Cond \rangle \} \langle R_d \rangle, \langle CPSR/SPSR \rangle$

$MSR \{ \langle Cond \rangle \} \langle CPSR/SPSR \rangle_{\langle fields \rangle}, R_m$

$MSR \{ \langle Cond \rangle \} \langle CPSR/SPSR \rangle_{\langle fields \rangle}, \#immediate$

In the syntax you can see a label called fields. This can be any combination of control (c), extension (x), status (s), and flags (f).

The CPSR and SPSR are not memory-mapped or part of the central register file. The only instructions which operate on them are the MSR and MRS instructions. These instructions are disabled when the CPU is in USER mode.

Coprocessor Instructions :

Coprocessor are used to extend the instruction set. A coprocessor can either provide additional computation capability or be used to control the memory subsystem including caches and memory management. The Coprocessor instructions include data processing, register transfer, and memory-transfer instructions.

Syntax :

CDP { <Cond> } Cp, opcode1, Cd, Cn [, opcode2]
 <MRC|MCR> { <Cond> } Cp, opcode1, Rd, Cn, Cn [, opcode]
 <LDC|STC> { <Cond> } Cp, Cd, addressing

CDP	Coprocessor data processing - perform an operation in a Coprocessor.
MRC MCR	Coprocessor register transfer - move data to/from Coprocessor registers.
LDC STC	Coprocessor memory transfer - Load and store blocks of memory to/from a Coprocessor.

In the syntax of the Coprocessor instructions, the Cp field represents the Coprocessor number between P_0 and P_{15} . The opcode fields describe the operation to take place on the Coprocessor. The C_n , C_m , and C_d fields describe registers within the Coprocessor.

The Coprocessor operations and registers depend on the specific Coprocessor you are using. Coprocessor 15 (CP15) is reserved for system control purposes, such as memory management, write buffer control, cache control and identification registers.

Loading Constants :

You might have noticed that there is no ARM instruction to move a 32-bit constant into a register. Since ARM instructions are 32-bits in size, they obviously cannot specify a general 32-bit constant.

To aid programming there are two pseudo instructions to move a 32-bit value into a register.

Syntax: LDR $R_d, = \text{Constant}$
ADR R_d, Label

LDR	Load Constant pseudoinstruction $R_d = 32\text{-bit constant}$
ADR	Load address pseudoinstruction $R_d = 32\text{-bit relative address.}$

The first pseudoinstruction writes a 32-bit constant to a register using whatever instructions are available. It defaults to a memory read if the constant cannot be encoded using other instructions.

The second pseudo instruction writes a relative address into a register, which will be encoded using a pc-relative expression.